Pushdown Automata: Definition Formal Definition of Pushdown Automata, A Graphical Notation for PDA's, Instantaneous Descriptions of a PDA,

Languages of PDA: Acceptance by Final State, Acceptance by Empty Stack, From Empty Stack to Final State, From Final State to Empty Stack

Equivalence of PDA's and CFG's: From Grammars to Pushdown Automata, From PDA's to Grammars

Deterministic Pushdown Automata: Definition of a Deterministic PDA, Regular Languages and Deterministic PDA's, DPDA's and Context-Free Languages, DPDA's and Ambiguous Grammars

Properties of Context-Free Languages: Normal Forms for Context-Free Grammars, The Pumping Lemma for Context-Free Languages, Closure Properties of Context-Free Languages, Decision Properties of CFL's

Module –IV **(10 Lectures)**

Introduction to Turing Machines: The Turing Machine: The Instantaneous Descriptions for Turing Machines, Transition Diagrams for Turing Machines, The Language of a Turing Machine, Turing Machines and Halting

Programming Techniques for Turing Machines, Extensions to the Basic Turing Machine, Restricted Turing Machines, Turing Machines and Computers,

Undecidability: A Language That is Not Recursively Enumerable, Enumerating the Binary Strings, Codes for Turing Machines, The Diagonalization Language

An Undecidable Problem That Is RE: Recursive Languages, Complements of Recursive and RE languages, The Universal Languages, Undecidability of the Universal Language

Undecidable Problems About Turing Machines: Reductions, Turing Machines That Accept the Empty Language. Post's Correspondence Problem: Definition of Post's Correspondence Problem, The "Modified" PCP, Other Undecidable Problems: Undecidability of Ambiguity for CFG's

Text Book:

1. Introduction to Automata Theory Languages, and Computation, by J.E.Hopcroft, R.Motwani & J.D.Ullman (3rd Edition) – Pearson Education
2. Theory of Computer Science (Automata Language & Computations), by K.L.Mishra & N. Chandrashekhar, PHI

MODULE-I

What is TOC?

In theoretical computer science, the **theory of computation** is the branch that deals with whether and how efficiently problems can be solved on a model of computation, using an algorithm. The field is divided into three major branches: automata theory, computability theory and computational complexity theory.

In order to perform a rigorous study of computation, computer scientists work with a mathematical abstraction of computers called a model of computation. There are several models in use, but the most commonly examined is the Turing machine.

Automata theory

In theoretical computer science, **automata theory** is the study of abstract machines (or more appropriately, abstract 'mathematical' machines or systems) and the computational problems that can be solved using these machines. These abstract machines are called automata.

This automaton consists of

- **states** (represented in the figure by circles),
- and **transitions** (represented by arrows).

As the automaton sees a symbol of input, it makes a *transition* (or *jump*) to another state, according to its ***transition function*** (which takes the current state and the recent symbol as its inputs).

Uses of Automata: compiler design and parsing.

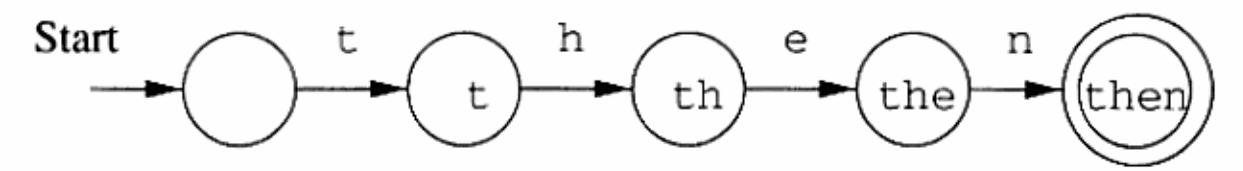

Figure 1.2: A finite automaton modeling recognition of **then**

Introduction to formal proof:

Basic Symbols used :

U – Union

∩- Conjunction

□ - Empty String

Φ – NULL set

7- negation

‘ – compliment

= > implies

Additive inverse: a+(-a)=0
Multiplicative inverse: a*1/a=1
Universal set U={1,2,3,4,5}
Subset A={1,3}
A' ={2,4,5}
Absorption law: AU(A ∩B) = A, A∩(AUB) = A

De Morgan's Law:
(AUB)' =A' ∩ B'
(A∩B)' = A' U B'
Double compliment
(A')' =A
A ∩ A' = Φ

Logic relations:
a → b = > **7**a U b
7(a∩b)=**7**a U **7**b

Relations:
Let a and b be two sets a relation R contains aXb.
Relations used in TOC:
Reflexive: a = a
Symmetric: aRb = > bRa
Transition: aRb, bRc = > aRc
If a given relation is reflexive, symmentric and transitive then the relation is called equivalence relation.

Deductive proof: Consists of sequence of statements whose truth lead us from some initial statement called the hypothesis or the give statement to a conclusion statement.

The theorem that is proved when we go from a hypothesis H to a conclusion C is the statement "if H then C." We say that C is *deduced* from H.

Additional forms of proof:
Proof of sets
Proof by contradiction
Proof by counter example

Direct proof (AKA) Constructive proof:
If *p* is true then *q* is true
Eg: if a and b are odd numbers then product is also an odd number.
Odd number can be represented as 2n+1
a=2x+1, b=2y+1
product of a X b = (2x+1) X (2y+1)
= 2(2xy+x+y)+1 = 2z+1 (odd number)

Proof by contrapositive:

The *contrapositive* of the statement "if H then C" is "if not C then not H." A statement and its contrapositive are either both true or both false, so we can prove either to prove the other.

Theorem 1.10: $R \cup (S \cap T) = (R \cup S) \cap (R \cup T)$.

	Statement	Justification
1.	x is in $R \cup (S \cap T)$	Given
2.	x is in R or x is in $S \cap T$	(1) and definition of union
3.	x is in R or x is in both S and T	(2) and definition of intersection
4.	x is in $R \cup S$	(3) and definition of union
5.	x is in $R \cup T$	(3) and definition of union
6.	x is in $(R \cup S) \cap (R \cup T)$	(4), (5), and definition of intersection

Figure 1.5: Steps in the "if" part of Theorem 1.10

	Statement	Justification
1.	x is in $(R \cup S) \cap (R \cup T)$	Given
2.	x is in $R \cup S$	(1) and definition of intersection
3.	x is in $R \cup T$	(1) and definition of intersection
4.	x is in R or x is in both S and T	(2), (3), and reasoning about unions
5.	x is in R or x is in $S \cap T$	(4) and definition of intersection
6.	x is in $R \cup (S \cap T)$	(5) and definition of union

Figure 1.6: Steps in the "only-if" part of Theorem 1.10

To see why "if H then C" and "if not C then not H" are logically equivalent, first observe that there are four cases to consider:

1. H and C both true.

2. H true and C false.

3. C true and H false.

4. H and C both false.

Proof by Contradiction:

H and not C implies falsehood.

That is, start by assuming both the hypothesis H and the negation of the conclusion C. Complete the proof by showing that something known to be false follows logically from H and not C. This form of proof is called *proof by contradiction.*

It often is easier to prove that a statement is not a theorem than to prove it *is* a theorem. As we mentioned, if S is any statement, then the statement "S is not a theorem" is itself a statement without parameters, and thus can

Be regarded as an observation than a theorem.

Alleged Theorem 1.13: All primes are odd. (More formally, we might say: if integer x is a prime, then x is odd.)

DISPROOF: The integer 2 is a prime, but 2 is even. □

For any sets a,b,c if a∩b = Φ and c is a subset of b the prove that a∩c =Φ
Given : a∩b=Φ and c subset b
Assume: a∩c ≠ Φ
Then $\forall x,\ x\epsilon a$ and $x\epsilon c$ => $x\epsilon b$
= > a∩b ≠Φ = > a∩c=Φ(i.e., the assumption is wrong)

Proof by mathematical Induction:

Suppose we are given a statement $S(n)$, about an integer n, to prove. One common approach is to prove two things:

1. The *basis*, where we show $S(i)$ for a particular integer i. Usually, $i = 0$ or $i = 1$, but there are examples where we want to start at some higher i, perhaps because the statement S is false for a few small integers.
2. The *inductive step*, where we assume $n \geq i$, where i is the basis integer, and we show that "if $S(n)$ then $S(n+1)$."

- ***The Induction Principle***: **If we prove $S(i)$ and we prove that for all $n \geq i$, $S(n)$ implies $S(n+1)$, then we may conclude $S(n)$ for all $n \geq i$.**

Languages :

The languages we consider for our discussion is an abstraction of natural languages. That is, our focus here is on formal languages that need precise and formal definitions. Programming languages belong to this category.

Symbols :

Symbols are indivisible objects or entity that cannot be defined. That is, symbols are the atoms of the world of languages. A symbol is any single object such as ♠, a, 0, 1, #, begin, or do.

Alphabets :

An alphabet is a finite, nonempty set of symbols. The alphabet of a language is normally denoted by Σ. When more than one alphabets are considered for discussion, then subscripts may be used (e.g. Σ_1, Σ_2 etc) or sometimes other symbol like G may also be introduced.

Example :

$$\Sigma = \{0, 1\}$$
$$\Sigma = \{a, b, c\}$$
$$\Sigma = \{a, b, c, \&, z\}$$
$$\Sigma = \{\#, \nabla, \spadesuit, \beta\}$$

Strings or Words over Alphabet :

A string or word over an alphabet Σ is a finite sequence of concatenated symbols of Σ.

Example : 0110, 11, 001 are three strings over the binary alphabet { 0, 1 } .

aab, abcb, b, cc are four strings over the alphabet { a, b, c }.

It is not the case that a string over some alphabet should contain all the symbols from the alphabet. For example, the string cc over the alphabet { a, b, c } does not contain the symbols a and b. Hence, it is true that a string over an alphabet is also a string over any superset of that alphabet.

Length of a string :
The number of symbols in a string w is called its length, denoted by |w|.

Example : | 011 | = 4, |11| = 2, | b | = 1

Convention : We will use small case letters towards the beginning of the English alphabet to denote symbols of an alphabet and small case letters towards the end to denote strings over an alphabet. That is, $a, b, c \in \Sigma$ (symbols) and u, v, w, x, y, z are strings.

Some String Operations :
Let $x = a_1a_2a_3 \in a_n$ and $y = b_1b_2b_3 \in b_m$ be two strings. The concatenation of x and y denoted by xy, is the string $a_1a_2a_3 \cdots a_nb_1b_2b_3 \cdots b_m$. That is, the concatenation of x and y denoted by xy is the string that has a copy of x followed by a copy of y without any intervening space between them.

Example : Consider the string 011 over the binary alphabet. All the prefixes, suffixes and substrings of this string are listed below.

Prefixes: ε, 0, 01, 011.
Suffixes: ε, 1, 11, 011.
Substrings: ε, 0, 1, 01, 11, 011.

Note that x is a prefix (suffix or substring) to x, for any string x and ε is a prefix (suffix or substring) to any string.

A string x is a proper prefix (suffix) of string y if x is a prefix (suffix) of y and x ≠ y.

In the above example, all prefixes except 011 are proper prefixes.

Powers of Strings : For any string x and integer $n \geq 0$, we use x^n to denote the string formed by sequentially concatenating n copies of x. We can also give an inductive definition of x^n as follows:
x^n = e, if n = 0 ; otherwise $x^n = xx^{n-1}$

Example : If $x = 011$, then $x^3 = 011011011$, $x^1 = 011$ and $x^0 = e$

Powers of Alphabets :

We write Σ^k (for some integer k) to denote the set of strings of length k with symbols from Σ. In other words,

Σ^k = { w | w is a string over Σ and | w | = k}. Hence, for any alphabet, Σ^0 denotes the set of all strings of length zero. That is, Σ^0 = { e }. For the binary alphabet { 0, 1 } we have the following.

$\Sigma^0 = \{e\}$.

$\Sigma^1 = \{0, 1\}$.

$\Sigma^2 = \{00, 01, 10, 11\}$.

$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

The set of all strings over an alphabet Σ is denoted by Σ^*. That is,

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \cdots \Sigma^n \cup \cdots$$
$$= \cup \Sigma^k$$

The set Σ^* contains all the strings that can be generated by iteratively concatenating symbols from Σ any number of times.

Example : If Σ = { a, b }, then Σ^* = { ε, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, …}.

Please note that if $\Sigma = F$, then Σ^* that is $\phi^* = \{e\}$. It may look odd that one can proceed from the empty set to a non-empty set by iterated concatenation. But there is a reason for this and we accept this convention

The set of all nonempty strings over an alphabet Σ is denoted by Σ^*. That is,

$$\Sigma^+ = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \cdots \Sigma^n \cup \cdots$$
$$= \cup \Sigma^k$$

Note that Σ^* is infinite. It contains no infinite strings but strings of arbitrary lengths.

Reversal :

For any string $w = a_1 a_2 a_3 \cdots a_n$ the reversal of the string is $w^R = a_n a_{n-1} \cdots a_3 a_2 a_1$.

An inductive definition of reversal can be given as follows:

Languages :
A language over an alphabet is a set of strings over that alphabet. Therefore, a language L is any subset of Σ^*. That is, any $L \subseteq \Sigma^*$ is a language.

Example :

1. F is the empty language.
2. Σ^* is a language for any Σ.
3. {e} is a language for any Σ. Note that, $\phi \neq \{e\}$. Because the language F does not contain any string but {e} contains one string of length zero.
4. The set of all strings over { 0, 1 } containing equal number of 0's and 1's.
5. The set of all strings over {a, b, c} that starts with a.

Convention : Capital letters A, B, C, L, etc. with or without subscripts are normally used to denote languages.

Set operations on languages : Since languages are set of strings we can apply set operations to languages. Here are some simple examples (though there is nothing new in it).

Union : A string $x \in L_1 \cup L_2$ iff $x \in L_1$ or $x \in L_2$

Example : { 0, 11, 01, 011 } $\cup$ { 1, 01, 110 } = { 0, 11, 01, 011, 111 }

Intersection : A string, x $\in$ $L_1 \cap L_2$ iff x $\in$ L_1 and x $\in$ L_2 .

Example : { 0, 11, 01, 011 } $\cap$ { 1, 01, 110 } = { 01 }

Complement **:** Usually, Σ^* is the universe that a complement is taken with respect to.
Thus for a language L, the complement is L(bar) = { $x \in \Sigma^*$ | $x \notin L$ }.

Example : Let L = { x | |x| is even }. Then its complement is the language { $x \in \Sigma^*$ | $|x|$ is odd }.
Similarly we can define other usual set operations on languages like relative complement, symmetric difference, etc.

Reversal of a language :
The reversal of a language L, denoted as L^R, is defined as: $L^R = \{w^R \mid w \in L\}$.

Example :

1. Let L = { 0, 11, 01, 011 }. Then L^R = { 0, 11, 10, 110 }.

2. Let L = { 1^n0^n | n is an integer }. Then L^R = { 1^n0^n | n is an integer }.

Language concatenation : The concatenation of languages L_1 and L_2 is defined as $L_1 L_2$ = { xy | $x \in L_1$ and $y \in L_2$ }.

Example : { a, ab }{ b, ba } = { ab, aba, abb, $abba$ }.

Note that ,

1. $L_1 L_2 \neq L_2 L_1$ in general.
2. $L\Phi = \Phi$
3. $L\{\varepsilon\} = L = \{\varepsilon\}$

Iterated concatenation of languages : Since we can concatenate two languages, we also repeat this to concatenate any number of languages. Or we can concatenate a language with itself any number of times. The operation L^n denotes the concatenation of L with itself n times. This is defined formally as follows:

$$L_0 = \{e\}$$
$$L^n = LL^{n-1}$$

Example : Let L = { a, ab }. Then according to the definition, we have

$$L_0 = \{e\}$$
$$L_1 = L\{e\} = L = \{a, ab\}$$
$$L_2 = L\,L_1 = \{a, ab\}\{a, ab\} = \{aa, aab, aba, abab\}$$
$$L_3 = L\,L_2 = \{a, ab\}\{aa, aab, aba, abab\}$$
$$= \{aaa, aaab, aaba, aabab, abaa, abaab, ababa, ababab\}$$

and so on.

Kleene's Star operation : The Kleene star operation on a language L, denoted as L^* is defined as follows :

L^* = (Union n in N) L^n

$= L^0 \cup L^1 \cup L^2 \cup \cdots$

= { x | x is the concatenation of zero or more strings from L }